Fusing the DeepSeek-R1 MoE Expert Region on MI355X

A HipKittens dispatch $\rightarrow$ GEMM $\rightarrow$ combine fusion: a prefill win, and what decode taught us

Subha

AMD · advised by Simran Arora & Muhammad Osama

DeepSeek-R1 MoE · 8×MI355X (CDNA4 / gfx950) · built on HipKittens + IRIS

June 30, 2026

The one-slide story

- *Problem.* In the production DeepSeek-R1 serving config, the MoE expert region is a chain of *separate* library kernels (MORI dispatch → aiter fmoe → MORI combine). The expert GEMM is good; the *handoffs around it* are not.
- *Thesis. aiter fuses the expert math; we fuse the expert region* — make the cross-GPU communication produce the GEMM layout directly, fuse activation/requant, and replace scatter-combine with a bandwidth-efficient pull-combine. Built on HipKittens (CDNA4) + IRIS (XGMI).
- *Result (8 full MI355X, same-node, correctness-gated).*
  - *Prefill: 1.56× faster* than the unfused MORI+aiter baseline (1247 vs 1941 μs) — and our pull-combine *beats AMD's own MORI EpCombine* (386 vs 398 μs).
  - *Decode: a marginal ∼2–3% win at matched fp8 precision.*

Where the MoE expert region lives

After routing picks experts, with `topk=8` each token becomes up to 8 routed (token, expert) rows. Each row crosses to the GPU owning its expert, runs that expert's FFN, and is combined back.

One expert FFN (DeepSeek-R1: H=7168, INTER=2048, 256 experts, EP8 = 32 local)

```
x[H=7168]
-> fc1 / W13:  H -> 2*INTER=4096  (emits gate||up in one GEMM)
-> SiLU(gate) * up  +  fp8 re-quant (per-128 block scale)
-> fc2 / W2:  INTER=2048 -> H=7168
```

- `fc1` and `fc2` are the *two matmuls inside one expert*, not two MoE layers.
- In Expert Parallel (EP), the cross-GPU *dispatch* (gather routed rows to expert owners) sits *before* the GEMM, and the *combine* (scatter weighted outputs back) sits *after*. *That dispatch/combine is the new all-to-all that replaces TP's*

Step 0 — profiling setup (how we isolated the MoE comm)

New Profiling Setup

- C4: Splits 8x MI355X node w/ TP=4 and DP=2
  - Enables EP and uses DP attention. We use ATOM's default MORI backend as our primary optimization baseline
  - I also tried using the same set up but overriding MORI to execute with the low latency pathway (ALL2ALL=low-latency)
- C6: DP=8, TP =1
  - Eliminates all_reduce/reduce_scatter collectives related to weight/activation sharding
  - Control test so the only cross GPU communication happening during the forward pass is the MoE all to all communication
- Ran benchmark at concurrency=16

- *C4 = TP4 × DP2 + EP + DP-attention.* This is the real serving config; it is the *only* mode where the MoE all-to-all (`EpDispatch`/`EpCombine`) fires — it requires `dp_size>1` *and* `--enable-expert-parallel`. We use ATOM's default MORI backend as the *primary baseline*, and also probed MORI's low-latency all2all path.
- *C6 = DP8, TP1.* Control: removes the TP all-reduce/reduce-scatter entirely, so the *only* cross-GPU traffic in the forward pass is the MoE all-to-all. Confirms the gather is an EP-flag artifact, not TP noise.
- Benchmarked at concurrency 16 (decode-heavy).

Step 0 — profiling result: comm is 14–16% of *all* GPU time

Profiling Results

- Across all 3 variations, the MoE communication kernels (EpDispatch – gather, EPCombine – scatter) consume 14-16% of the total GPU kernel time
- Total target surface would be gather/scatter (14.23%) + adjacent sorting/quant (10.32%) = 24.55% of the decode

Category	C4 HT	C6 HT	C4 low-lat
dense GEMM	29.25%	28.79%	28.86%
MoE expert GEMM	26.80%	26.82%	26.82%
quant	12.55%	12.22%	12.26%
EP combine / scatter	**8.05%**	**8.32%**	**8.22%**
EP dispatch / gather	**6.18%**	**7.30%**	**7.21%**
gather + scatter	**14.23%**	**15.62%**	**15.44%**
TP collective	~0	**0.00%**	**0.00%**
gather+pack+quant envelope	24.55%	~24%	25.50%

Stable across all three variants (C4 HT, C6 HT, C4 low-lat):

- *gather + scatter = 14–16%* of total GPU kernel time (`EpDispatch` ∼6–7% + `EpCombine` ∼8%).
- Add the *adjacent*, fusable sorting + quant (∼10%) ⇒ a *∼24.5% "gather + pack + quant" envelope* of the decode step.
- The expert GEMM itself is ∼27%; dense (non-MoE) GEMM ∼29%.

The target surface

The fusable envelope — everything *around* the expert GEMM — is a quarter of the decode step. That is what we attack.

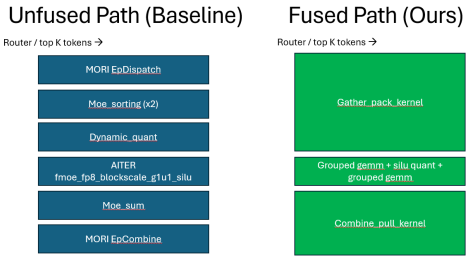
Why decode is a *weight* wall — and why that *caps* the decode win

Key term: M_e = rows (token–expert pairs) an expert processes. At *decode* you generate only a handful of tokens per step; with top-8 routing over 256 experts (32 local/GPU), even a 64-token step lands ∼16 rows on each local expert. Sweeping the stock region over batch size:

batch (tokens in flight)	rows/expert (M_e)	region μs
decode (∼16 tokens)	4	291.7
decode (∼64 tokens)	16	329.1
mid (∼256 tokens)	64	568.7
prefill (∼1024 tokens)	256	1416.7

- *The expert GEMM reads all 32 local experts' ∼1.4 GB of fp8 weights from HBM every step* — a fixed ∼176 μs floor (8 TB/s), *regardless of token count*. The region barely moves (292→329) as tokens drop: it has hit that floor.
- *Decode region ≈ 80% weight floor + 20% boundaries* (gather/sort/quant/combine). That floor is *identical* for us and the baseline (same fp8 weights), so fusion can't touch it ⇒ *even if every boundary were free, decode caps at ∼1.25×* ($= 1/0.8$); we land ∼1.03×.
- *Prefill has no such cap* — its GEMM does token-scaled compute (not a fixed floor), so the *whole* region is improvable → 1.56×. *(So "shouldn't decode win more?" — no: a bigger boundary fraction can't beat the weight wall that dominates decode.)*

The unfused baseline, drawn precisely



`aiter.fused_moe` is *one Python call* but *many GPU kernels*. The full C4-faithful region is *seven stages*:

1. MORI `EpDispatch` (XGMI gather)
2. `moe_sorting` pass 1 (counts/offsets)
3. `moe_sorting` pass 2 (scatter to expert order)
4. `dynamic_quant` (bf16 $\rightarrow$ fp8 + scales)
5. `fmoe_..._g1u1_silu` (*the good kernel*: fc1+SiLU+fc2)
6. `moe_sum` (weighted top-k reduce)
7. MORI `EpCombine` (XGMI scatter)

The inefficiency

A *good* expert kernel surrounded by *expensive layout / materialization boundaries*.

Why the unfused path bleeds: every boundary is an HBM round-trip

The expert kernel is *not* the problem — the stages *around* it each read and re-write activations to HBM in a *different* representation:

- *dispatch → sort:* tokens arrive in *communication* order, but the GEMM needs *expert-major* order. Sorting (2 passes) is pure layout traffic + sync — *not* GEMM work.
- *sort → quant:* C4-faithful dispatch moves *bf16* tokens; fp8 fmoe needs fp8+scales. `dynamic_quant` is a full read(bf16)/write(fp8) pass over the activation buffer.
- *fmoe → combine:* fmoe writes expert outputs to HBM; `moe_sum`/`EpCombine` read them back to accumulate weighted top-k and ship to origin ranks.
- *launch/sync overhead:* most visible at *decode* (tiny rows, fixed costs dominate).

Takeaway for the slide

Unfused cost = good expert kernel + standalone *sort* + standalone *quant* + scatter *combine*, each paying a real HBM read/write at the boundary. *None of that is matmul.*

What we actually built — fuse the *region*, not the math

Unfused (7 stages):

```
EpDispatch
 -> moe_sorting x2
 -> dynamic_quant
 -> fmoe[fc1 + SiLU*up + fc2]
 -> moe_sum
 -> EpCombine
```

Ours (region-fused):

```
gather_pack [IRIS]
 -> grouped_b0 fc1 [HK]
 -> silu_quant [HK]
 -> grouped_b0 fc2 [HK]
 -> combine_pull [IRIS]
```

- *[HK] HipKittens* does the on-GPU compute (fc1/fc2 GEMM, SiLU+requant, the local reduce); *[IRIS]* does the cross-GPU communication (the gather pull, the combine write). *No aiter* — its `fmoe` is the baseline we replace.
- This is the design from our meeting: *HipKittens kernels for the math, an IRIS load/store on the MoE side for the comm.*
- *We own the intermediate layouts*, so the comm *produces* the GEMM's input layout — killing the standalone sort, the quant pass, and the scatter combine.

Kernel 1 — `gather_pack` (IRIS): the communication *is* the sort

Replaces MORI `EpDispatch` + the two `moe_sorting` passes. For each routed row we look up its source `(rank, row)` from the routing metadata, then issue *one IRIS `ctx.load(ptr, src_rank)`* — a direct XGMI read of that row's 16 fp8 bytes + its fp32 scale — straight into our *expert-major* packed buffer. Rows sourced from this same GPU skip XGMI (plain local load).

- *No sort pass:* the slot we write *is* the expert-major row the GEMM wants — communication and layout become one pass.
- *"Zero sentinel" = a poison-free pad.* To run all 32 experts in one launch each expert's row-block is padded; padding/unrouted rows are written as fp8 0x00 with scale 0, so they multiply to *exactly zero* in the GEMM — a padded row can never corrupt a real expert's output.
- *Comm/compute overlap, demonstrated (the meeting's goal).* In an overlap prototype we folded the fp8→bf16 dequant *into* this gather so it runs while the XGMI loads are still in flight: *gather+dequant 162 μs* vs *gather-only 168 μs* — the dequant is *hidden*, i.e. free. (The shipped region runs the dequant as a separate vectorized kernel; the harder *in-GEMM* overlap is on the "what didn't work" slide.)

Kernel 2 — the expert GEMM `grouped_b0` (HipKittens)

The kernel that does the actual expert matmuls — *both* fc1 and fc2. It is our own HipKittens CDNA4 GEMM, not aiter:

- *8 waves, 256×256×64 tiles*, double-buffered shared→register *ping-pong*, `mma_ABt` (16×16×32 MFMA) into an fp32 accumulator — written in HipKittens tile types (`st_bf` shared, `rt_bf` register, `rt_fl` accumulator).
- *One thread-block per per-expert tile*, launched from a flat task list, so all 32 local experts compute in *one* launch (this is what replaces aiter's grouped `fmoe` GEMM).
- *It consumes bf16.* The gather's packed A is fp8, so a *vectorized dequant kernel* (`fp8x4`→`bf16`, coalesced) converts it right before the GEMM — the "dequant preamble." Vectorizing it (scalar → packed) cut ∼117 μs off fc1.
- We dequant to bf16 then MFMA, so *the prefill GEMM is bf16-precision* — that's where the clean RMS 0.018 comes from. (Decode swaps in a native-fp8 variant — next slide.)

Kernel 3 — the activation `silu_quant` (HipKittens) — *not* a matmul

This is the *middle* of the FFN, between fc1 and fc2: read fc1's output $C_1 = \text{gate}\|\text{up}$, compute $h = \mathrm{SiLU}(\text{gate}) \cdot \text{up}$, then *re-quantize h to fp8* (per-128 block scale = amax/448) so fc2 can consume it. No GEMM — just activation + requant.

- *Why a "PyTorch" baseline below, not the unfused path:* aiter does this activation *inside* its monolithic `fmoe`, so there is *no* standalone unfused activation to race. When we split the region we had to write it ourselves — and our first cut just called PyTorch's `silu_and_quant`. The table is the optimization of *our own* kernel:

our v1 (calls PyTorch silu+quant)	471 μs
fused into one kernel	127 μs
+ local-amax (16-way vs 128-way atomics)	*38.6 μs*

The big jump (127→38.6): the per-128 amax was *every* thread doing an `atomicMax` into shared memory — 128-way contention. Instead each thread keeps a *local* amax over its own contiguous slice and does *one* `atomicMax` (16-way). (At decode a task-driven variant that skips padded rows reaches 7.2 μs.)

Kernel 4 — `combine` (IRIS): beating MORI's own combine

The combine is the *reverse* of the gather — send each expert-output (fc2) row back to its origin token, scaled by the route weight, summing the up-to-8 expert contributions per token.

- *v1 — scatter.* One block owns one fc2 row; for each element it does an *IRIS atomic-add (`ctx.fetch_add`)* into the origin token's fp32 accumulator on the *origin rank* over XGMI. The atomic is needed because top-8 means several expert outputs land on the same token. $\Rightarrow$ *788 μs*.
- *Diagnosis — the bytes, not the atomics.* Dropping the atomic saved $\sim$4%; vectorizing the stores, 0%. It is *XGMI write-bandwidth bound*: $8192 \times 7168 \times 4\text{B} \approx$ *234 MB* of fp32 remote writes. The only lever is *fewer bytes*.
- *Fix — pull / gather-reduce.* Invert it: each *destination* token gathers its $\leq$8 fc2 rows from the *local* buffer, sums them in fp32 locally (*no cross-rank atomics*), and writes *one bf16* row home via `ctx.store` (234$\rightarrow$117 MB). The local fp32 sum is what makes a bf16 result correct under collisions.

pull, bf16, cells *sorted* by dst_rank	934 μs (*worse!*)
pull, bf16, cells round-robin'd across dst_rank	*386 μs*
MORI `EpCombine` (the baseline)	398 μs

Decisive: sorting destinations hammers one XGMI link at a time; *round-robin spreads concurrent blocks over all 8 links* — same bytes, 2.4$\times$ faster, below MORI.

The decode GEMM — a *different*, skinny BM=16 native-fp8 kernel

Decode swaps in a different GEMM than prefill. Prefill's `grouped_b0` uses 256-row tiles; at decode each expert has only ∼4–16 real rows, so a 256-row tile is ∼94% padding and the GEMM goes *compute-bound grinding zeros*. `grouped_b0_gemm_decode` uses a *skinny BM=16 tile* (the MFMA's minimum M) — that drops the padded compute *below* the weight-stream floor, so decode finally becomes weight-memory-bound, where the one lever is *fp8 weights* (half the HBM bytes).

- *Native fp8 with no fp8 load path:* HipKittens can't load fp8 to registers, so we *store fp8 but load as a half-width bf16 tile* (the fast path), unpack fp8→bf16 in-register, bf16×bf16 MFMA — weights *pre-swizzled offline* into fragment order. Standalone: *3.97 TB/s, 1.46× over bf16*, RMS 0.0037 — matches aiter's fused fp8 (decode 171.0 vs 171.6 TFLOP/s).
- *A subtle bug (14 experiments):* a stable wrong answer (RMS ≈ 0.71) + an apparent hang. *Not a data bug — a compiler scheduling hazard*: the async `ds_read`'s inline-asm output looked register-ready, so the compiler hoisted the first `mma` *above* the `s_waitcnt` that waits for the data ⇒ it multiplied not-yet-arrived operands (garbage = $1/\sqrt{2}$); the bad read also computed out-of-bounds addresses = the "hang."
- *Fix:* a one-line scheduling barrier (`s_waitcnt lgkmcnt(0)` + `sched_barrier`) before the mma — both symptoms gone.

How we benchmarked (the part experts will poke at)

Compare regions, not kernels — define the region by its I/O boundary (enter: routed bf16 tokens + top-k; exit: combined bf16 outputs, token order) and measure *both* sides over it.

- *Same node, same operating point.* Nodes vary $\sim$1.8$\times$ in speed across the cluster — an early "decode 920 μs loss" was just a throttled node (its prefill was 2198 vs 1240). *Always run b_1 and b_3 back-to-back on one node.*
- *Pre-warm aiter.* aiter JIT-compiles its fmoe per (shape,dtype) into a *container-local* cache ($\sim$10 min cold) — a "hang" that is really a cold cache. Warm it before timing.
- *MAX over 8 ranks*, device-event timed, warmup + median, *correctness gate (RMS) before timing*, reported at *decode and prefill*.
- Baseline = stock MORI `EpDispatch` (bf16) + aiter `fused_moe` + MORI `EpCombine` — the C4-faithful production chain.

Result — PREFILL: 1.56× (the strong, clean claim)

8 full MI355X, same node (thor-4), correctness-gated:

region	fused b_1	unfused b_3	speedup
prefill (TOTAL_M=8192)	*1247 µs*	1941 µs	*1.56×*

Per-stage decomposition (illustrative; *don't mix denominators across nodes*):

stage	unfused	fused (ours)
dispatch / gather	290	*147*
expert FFN GEMM	2622 (fmoe)	921 (fc1) + 481 (fc2)
activation	— (in fmoe)	*39*
combine	398 (MORI)	*387* (pull)
region	**3246**	**1974** (1.64×)

Why prefill wins: the GEMM is bandwidth/compute-rich, so the eliminated boundaries (sort, quant materialization, scatter combine) are pure savings — *and* our combine beats MORI, our act is near-free, and the dequant hides under the gather. *Bonus: our prefill GEMM is bf16-dequant (→ RMS 0.018), higher precision than aiter's fp8.*

Result — DECODE: honest, marginal, and *why*

Same node (thor-4), warm aiter, correctness-gated:

region	fused b_1	unfused b_3	verdict
decode (TOTAL_M=512)	*516.5 μs*	527 μs (533 canon.)	$\sim$2–3% win

Per-stage: gather 41.5 + fc1 274 + *act 7.2* + fc2 150 + combine 47.

- *Why only $\sim$2–3%:* as the cost model predicted, decode is *weight-bound* — fc1+fc2 = 424 of 516 μs is just the two fp8 weight streams near the in-region HBM rate. The boundaries we removed were only $\sim$18% to begin with; we recovered the act tax (25$\rightarrow$7 μs) and edged ahead.
- *The RMS caveat (be precise).* The fp8-weight region scores RMS 0.057 vs a bf16-weight reference; bf16 weights would score 0.018. That 0.018$\rightarrow$0.057 jump *is* e4m3's $\sim$3.6% per-weight error (scale-invariant; finer scales don't help). *But the aiter baseline also runs fp8 weights* (`fmoe_bf16_blockscaleFp8`) — so 0.057 is production's *own* precision class. A strict RMS$<$0.05 needs bf16 on a matmul = +135 μs = a decode *loss*. It's a real μs-vs-precision tradeoff, and at *matched* precision decode wins.

What didn't work (1/2) — and the architectural reason

- *Host-stream overlap (gather ‖ GEMM on two HIP streams).* Even after fixing chunked-gather grid collapse, concurrent = −12 to −153 μs vs serial. *Reason:* the B0 GEMM already saturates the CUs, so two streams just time-slice. The expert GEMM is not idle enough to hide the gather at the *stream* level.
- *In-kernel gather-under-MFMA (the "V2" dream).* The only existing body that does remote gather under MFMA runs *35.8 vs 466 TFLOP/s — 13× slower*. *Reason:* the fast B0 body has *no* producer/consumer warp split and *no* A-reuse across N-subtiles; blocking remote loads stall the waves instead of hiding comm. This needs a *different* GEMM body (async A-fill + reuse), not a patch — a real open direction.

The honest thesis boundary

Comm/compute overlap *works* where compute sits in comm's shadow (the dequant-in-gather). It does *not* work as warp-level gather-under-MFMA on today's peak GEMM body — the GEMM is too busy and not structured for it.

What didn't work (2/2)

- *Native fp8 at BM=256.* +8% at full occupancy, neutral/negative at decode. *Reason:* at BM=256, decode is *compute-bound on padding*, so halving weight bytes doesn't help. fp8 only pays off once a small-BM tile makes decode weight-*memory*-bound.
- *Register double-buffer for the decode tile.* $\sim 3\times$ *slower*. *Reason:* doubling the A/B register tiles blew VGPR $\rightarrow$ occupancy collapse below `launch_bounds`; at BM=16, latency hiding = *occupancy* (many warps), not a wider per-thread pipe. LDS double-buffering (VGPR-cheap) was the right tool.
- *Compacting the GEMM padding (Mpacked 8192$\rightarrow$512).* Reassessed as *high-risk / low-reward*: the weight stream is a *contiguous* per-expert buffer, so the padding adds compute that largely hides under it; the recoverable cost was the *act* tax (already taken, 25$\rightarrow$7 μs). Compaction would also break the ERB % 256 alignment the gather/GEMM rely on (history: rewrites $\rightarrow$ RMS$\approx$1.0). *Profile before you pay.*
- *Strict RMS$<$0.05 decode win.* Fundamental: it requires a bf16 matmul = +135 μs = a loss. Not a bug to fix — a precision floor to state honestly.

Aside — the "padding tax," and is the comparison fair?

A natural expert question: *why does our grouped GEMM pad each expert to 256 rows (8192 packed for ∼512 real at decode)?*

- *Why we pad:* flattening 32 experts into one launch requires each expert's region rounded up to the tile height (BM), so a tile never straddles two experts (the "no-contamination" invariant). Our 8-wave body uses BM=256.
- *Does production have it too? Yes.* aiter's `moe_sorting` pads each expert to its tile granularity (`block_m` ∼16–32). *Padding is inherent to grouped/sorted MoE GEMM* — ours is just ∼8× coarser, a self-inflicted (fixable) handicap, *not* an unfair benchmark.
- *Does it cost wall-clock? Mostly no (we measured).* Because decode is weight-bound, the padding adds *compute* that hides under the weight stream — the one exposed cost was the *activation* kernel launching one block per padded row, which we fixed (act 25→7 μs). The GEMM-side compaction wasn't worth the alignment risk.

Summary — aiter fuses the math; we fuse the region

What the fusion buys (all measured, 8 full MI355X, pushed to GitHub):

- *Prefill 1.56×* over the production MORI+aiter chain — the headline.
- A pull-combine *below MORI's own* EpCombine (386 vs 398 μs).
- Near-free activation (471→7–39 μs) and *dequant hidden under the XGMI gather* (comm/compute overlap, demonstrated).
- Decode brought from a loss to a *marginal win at production-matched fp8 precision*.

The honest boundaries:

- Decode is a *weight* wall — fusion's ceiling there is ∼1.1–1.4×, and the fp8 RMS floor is real.
- Warp-level gather-under-MFMA needs a new GEMM body (open work).
- Bigger decode levers are orthogonal: more EP shards, fp4 weights, larger batch.

Substrate: HipKittens (CDNA4 8-wave MFMA, fp8/bf16 tiles, `sched_barrier`) + IRIS (XGMI `ctx.load`).

One line

Make the communication produce the compute layout, hide the conversions in its shadow, and reduce on *pull* not scatter — and the fused expert region beats the unfused production baseline, decisively at prefill.

Backup — numbers at a glance

quantity	ours	baseline
prefill region (8192)	1247 μs	1941 μs (1.56×)
decode region (512), warm same-node	516.5 μs	527 μs
combine	386 μs (pull)	398 μs (MORI)
activation	38.6 / 7.2 μs	— (inside fmoe)
dequant (folded in gather)	≈0 (162 vs 168 gather-only)	full pass
decode GEMM (standalone)	3.97 TB/s, 1.46× over bf16, RMS 0.0037	aiter parity
prefill RMS / decode RMS	0.018 / 0.057	(fp8 class)
scatter combine (rejected)	788 μs (write-BW-bound, 234 MB)	
in-kernel gather-under-MFMA (rejected)	35.8 vs 466 TFLOP/s	
host-stream overlap (rejected)	−12 to −153 μs	

Repo: `github.com/subha-amd/iris` (branch `subha/moe-dispatch-v0`). Decode is weight-bound; gather/combine = ∼18% of the decode region, ∼14–16% of total GPU time.